

clautolisp Command Specifications

Pascal Bourguignon

May 24, 2026

Contents

1	Scope	1
2	Synopsis	1
3	Modes of Operation	2
3.1	File mode	2
3.2	Expression mode	2
3.3	Action queue	3
3.4	Interactive REPL	3
4	Options	4
4.1	Dialect selection	4
4.2	Input selection	4
4.3	REPL behaviour	4
4.4	Verbosity	5
4.5	Informational	5
4.6	Unknown options	5
5	Dialects	5
6	Encoding and line endings	6
7	Exit Codes	6
8	Errors and Diagnostics	7
8.1	Reader errors	7
8.2	Runtime errors	7
8.3	Termination	8
8.4	Caught errors that escape	8

9 Builtins and Special Operators	8
10 Backtrace	8
11 File Compatibility	9
12 Building the Executable	9
13 User init files	9
13.1 Stem list	9
13.2 Per-stem extension preference	10
13.3 Gating	10
14 Versioning	11
15 Conformance	11
16 See Also	11

1 Scope

This document specifies the behaviour of the standalone `clautolisp` command, distributed as two interchangeable binaries:

- `clautolisp-sbcl` — the SBCL-built executable;
- `clautolisp-ccl` — the CCL-built executable.

Both are produced from the same source under `clautolisp/tools/clautolisp/` and present an identical command-line interface. Subsequent sections refer to either as `clautolisp`.

The AutoLISP / Visual LISP language semantics that the command evaluates are **not** re-specified here. They live in `autolisp-spec/documentation/autolisp-visual-lisp-spec` and are referenced from this document as "the AutoLISP Spec". The `clautolisp`-specific operators that go beyond that spec are listed in `clautolisp/autolisp-builtins-core/README` §"clautolisp-Specific Operators" and in `clautolisp/documentation/user-manual.org` §"clautolisp Extensions"; they are not redefined here either.

2 Synopsis

```
clautolisp [OPTIONS] FILE.lsp                # file mode (positional)
```

```

clautolisp [OPTIONS] -l FILE.lsp           # file mode (explicit)
clautolisp [OPTIONS] -x EXPRESSION         # expression mode
clautolisp [OPTIONS]                       # interactive REPL
clautolisp [OPTIONS] {-l FILE | -x EXPR} -i # action followed by REPL

```

Short and long forms of every option are interchangeable; the short forms listed below match the convention shared with the sibling **alfe** front-end:

Short	Long	Meaning
-l	--load	Load and evaluate the file at FILE .
-x	--eval	Evaluate the EXPRESSION argument as a source snippet.
-i	--interactive	Enter the REPL after the file/expression action (if any).
-q	--quiet	Suppress the REPL banner.
-v	--verbose	Print extra diagnostic information.
-d	--debug	Print debug traces and CL backtraces on runtime errors.
-h	--help	Print the option summary and exit 0.
-V	--version	Print the version and exit 0.

The argument vector seen by **clautolisp** is forwarded verbatim from the host shell. Both SBCL- and CCL-saved cores set **:save-runtime-options t** so the underlying Lisp does not intercept **--help**, **--version** or other runtime flags.

3 Modes of Operation

Exactly one of three primary modes is selected by the argument vector. When **-i** / **--interactive** is combined with an action, the action runs first and the REPL takes over its evaluation context.

3.1 File mode

A positional argument or **-l FILE** / **--load FILE** is interpreted as a path to an AutoLISP source file. The file is opened, decoded under the active reader options (see 5), every form is read and evaluated in order, and the value of the last form is discarded. Side effects are visible.

If the file cannot be opened, **clautolisp** exits with status 2 and writes a one-line diagnostic to standard error.

3.2 Expression mode

`-x` `EXPRESSION` (also `--eval` `EXPRESSION`) evaluates `EXPRESSION` instead of reading a file. `EXPRESSION` is treated as a source snippet: it may contain multiple top-level forms separated by whitespace; they are read and evaluated in order. The value of the last form is discarded. Each `-x` takes exactly one argument; absence of the argument is a usage error.

3.3 Action queue

Each `-l` / `--load`, `-x` / `--eval`, and positional `FILE` contributes one entry to an **action queue** in the order it appears on the command line. `clautolisp` builds a single fresh evaluation context and runs every queued action against that context, in order, so side effects compose: an early `-x` may define a function that a later `-l` then calls. When several input-selecting options are supplied, **all** of them run — there is no last-wins resolution.

If the queue is empty and `-i` is not given, `clautolisp` enters the REPL (the historical default with no args).

3.4 Interactive REPL

With no input-selecting option, `clautolisp` enters an interactive read-eval-print loop on standard input. With `-i` / `--interactive`, the REPL runs **after** the action queue (if any), inheriting the same evaluation context — so every definition made by the queued actions remains visible at the prompt. The loop reads one balanced top-level form (multiple input lines may be consumed when the form spans line breaks), evaluates it, and prints the result with a `prin1`-style formatter on standard output.

The loop exits cleanly when:

- standard input reaches EOF, OR
- a form invokes `(exit)`, `(quit)`, `(vl-exit-with-error ...)` or `(vl-exit-with-value ...)`.

Reader and evaluator errors do not terminate the loop: they are reported as one-line diagnostics on standard error and the prompt is reissued.

The default primary prompt is `_$ ~.` The continuation prompt issued when a partial form needs more input is three spaces. Both prompts are written to standard error so that `~clautolisp` output captured through `> file` remains free of prompt strings.

A startup banner is printed on standard error once, before the first prompt:

```
clautolisp <version> - REPL (<dialect> dialect) - Ctrl-D to exit.
```

The banner is suppressed by `--quiet`.

4 Options

Every option may appear at most once unless noted; later occurrences override earlier ones. Options may appear before or after the positional `FILE` argument; `--` is **not** recognised as end-of-options.

4.1 Dialect selection

`--dialect NAME` Select a named dialect. `NAME` must be one of `strict`, `autocad-2026`, `bricscad-v26`. Default: `strict`.

`--strict` Shorthand for `--dialect strict`.

`--autocad` Shorthand for `--dialect autocad-2026`.

`--bricscad` Shorthand for `--dialect bricscad-v26`.

The selected dialect determines reader token rules, string-escape handling, integer-overflow warning, hex-float parsing in `atof`, unbound-variable behaviour, and the implementation marker propagated to the runtime session. The descriptors are defined in `clautolisp/autolisp-reader/source/dialect.lisp`.

4.2 Input selection

`-l FILE`, `--load FILE` Append "load `FILE`" to the action queue. Each occurrence adds one entry; a positional `FILE` on the command line is treated the same way.

`-x EXPRESSION`, `--eval EXPRESSION` Append "evaluate `EXPRESSION`" to the action queue. The two spellings are exact synonyms. `EXPRESSION` may contain multiple top-level forms separated by whitespace.

`-i`, `--interactive` Enter the REPL after the action queue completes (or immediately, if the queue is empty). The REPL inherits the queue's evaluation context, so every definition made by the queued actions is visible at the prompt. Without `-i`, a non-empty queue runs to completion and the process exits.

4.3 REPL behaviour

-q, --quiet Suppress the REPL banner. Has no effect outside interactive mode. Short and long forms are equivalent.

4.4 Verbosity

-v, --verbose Enable extra diagnostic output. In REPL mode the banner gains a second line listing the active host and any non-default knobs (mock-input path, GUI command, trace flag). In file/expression mode, a one-line summary is printed to standard error when the action completes successfully. Off by default.

-d, --debug Enable debug-level diagnostics. Currently this adds the host Lisp's backtrace to the AutoLISP backtrace printed on a runtime error, so the underlying CL stack is available when debugging builtins or the runtime itself. Off by default. **--debug** implies **--verbose** for output purposes.

These two flags are antinomic when combined: **-v** followed by **-q** turns verbosity off; **-q** followed by **-v** leaves it on. Last occurrence wins. **-d** subsumes verbose-mode output regardless of order.

4.5 Informational

-V, --version Print the version string on standard output and exit 0. Format: `clautolisp <MAJOR.MINOR.DEVELOP>` where the version is taken from `clautolisp.tools.clautolisp:*version*`. The DEVELOP counter is bumped on every change to clautolisp source code, so the running binary's **--version** output is a stable identifier of the build.

-h, --help Print the option summary on standard output and exit 0.

4.6 Unknown options

Any unrecognised argument that begins with **-** aborts startup with exit status 1 and a one-line diagnostic on standard error.

5 Dialects

The dialect descriptor selected at startup propagates to:

- the reader, through reader options derived from the descriptor;
- the runtime session, through its `dialect` slot, queryable inside AutoLISP by `current-evaluation-dialect` and used by, e.g., the unbound-variable handler.

The descriptor knobs that the binary observes are listed in `clautolisp/autolisp-reader/source/`. Their semantic effects are documented in the AutoLISP Spec; this document does not duplicate them. The three named profiles are:

Dialect	Token mode	Extended escapes	Integer overflow warn	Hex-float atof	Unbound
<code>strict</code>	<code>strict</code>	<code>no</code>	<code>no</code>	<code>no</code>	<code>silent-NI</code>
<code>autocad-2026</code>	<code>strict</code>	<code>no</code>	<code>yes</code>	<code>no</code>	<code>silent-NI</code>
<code>bricscad-v26</code>	<code>lax</code>	<code>yes</code>	<code>yes</code>	<code>yes</code>	<code>silent-NI</code>

Encoding is **not** a dialect knob. It is selected at the input boundary by reader options (see 6).

6 Encoding and line endings

- File-mode input is decoded under the reader’s external-format setting. The default external format is determined at the input boundary (`autolisp-reader/source/input.lisp`). Source bytes that are valid in the chosen encoding are accepted; an unrecognised byte sequence aborts the read with a reader-error diagnostic.
- All input passes through the reader’s line-ending normalisation layer: `LF`, `CRLF` and `CR` are uniformly normalised to `LF` for source-position computation.
- Expression-mode input (the `EXPRESSION` argument of `-x`) is taken verbatim from the argument vector and treated as already decoded under the host shell’s encoding.
- REPL input is decoded under the host’s standard-input encoding.

7 Exit Codes

`clautolisp` exits with one of the following statuses:

- 0 successful evaluation of the file or expression; clean `--help` / `--version` output; clean termination of the REPL via EOF; a form that invoked `(exit)`, `(quit)` or `(vl-exit-with-value ...)`.
- 1 uncaught runtime error escaped from the AutoLISP evaluator; or an unknown command-line option; or any other unexpected internal failure that reaches the top-level handler.
- 2 the FILE argument could not be opened (reported as a CL `file-error` wrapped in a one-line diagnostic).

A non-zero exit code is the only programmatic signal of failure suitable for use from a Makefile or CI runner.

8 Errors and Diagnostics

`clautolisp` guarantees that the host Lisp's debugger is **never** entered: every error path terminates the process with a non-zero exit code after writing a one-line diagnostic on standard error.

8.1 Reader errors

Raised by the reader during file or expression mode, or by the REPL between forms. Format:

```
clautolisp: <severity>: <code>: <message>
  at <source-name>:<start-line>:<start-column>--<end-line>:<end-column>
```

The diagnostic is built from the structured `diagnostic` object the reader attaches to the `simple-error` condition. `severity` is one of `error`, `warning`, `note`. The source-name is the FILE path in file mode, "`<expression>`" in expression mode, and "`<repl>`" in interactive mode.

8.2 Runtime errors

Raised by the AutoLISP evaluator. Format:

```
clautolisp: runtime error: <code>: <message>
AutoLISP backtrace (most recent call first):
  in <KIND> <op>: <args>
  ...
```


`code` is the symbolic AutoLISP-runtime-error code (e.g. `UNDEFINED-FUNCTION`, `UNBOUND-VARIABLE`, `TYPE-ERROR`, `INVALID-NUMBER-ARGUMENT`). The back-trace section is included when the condition carries a captured call stack; it is omitted when the stack is empty (some legacy code paths in older builds did not capture it). Each frame names its kind (`:eval`, `:special-op`, `:subr`, `:usubr`) and the form or operator+args being evaluated at that point.

In file and expression modes, a runtime error escape exits 1. In the REPL, the error is reported and the next prompt is issued.

8.3 Termination

(`exit`) and (`quit`) raise an `autolisp-termination` condition that the binary intercepts:

```
clautolisp: terminated by <kind>
```

These are clean exits, status 0. They are **not** catchable by `vl-catch-all-apply`.

8.4 Caught errors that escape

`vl-exit-with-error` and `vl-exit-with-value` raise an `autolisp-namespace-exit` that is normally caught by an enclosing `vl-catch-all-apply`. If they reach the top level uncaught, the binary reports the carried value or message in a one-line diagnostic and exits 1 (`:error` kind) or 0 (`:value` kind).

9 Builtins and Special Operators

On startup, `clautolisp` installs the full core builtin registry into the freshly created runtime session. The registry covers every function listed in `clautolisp/autolisp-builtins-core` under `core-builtins`. The set is a **superset** of the AutoLISP Spec: the operators added by `clautolisp` are documented in `clautolisp/autolisp-builtins-core/README.org` §"clautolisp-Specific Operators" and in `clautolisp/documentation/user-manual.org` §"clautolisp Extensions" (the `Cxxxr/Cxxxxr` family beyond `CADR`, `ERROR`, `LCM`, `LOGXOR`, `VL-BT`, `VL-CATCH-ALL-ERROR-STACK`).

The implemented special operators are listed in `*special-operator-dispatch*` in `clautolisp/autolisp-runtime/source/api.lisp`. Their semantics are those of the AutoLISP Spec; this document does not redefine them.

10 Backtrace

The runtime tracks an AutoLISP-level call stack in the `*autolisp-call-stack*` dynamic variable. `autolisp-eval`, `eval-special-operator` and `call-autolisp-function-in-context` push and pop frames automatically. The captured stack is attached to every `autolisp-runtime-error` raised by the evaluator and the builtin layer; `clautolisp` renders it after the error message (see 8.2).

The `(vl-bt)` builtin prints the current stack at the call site (useful for instrumenting AutoLISP code without raising). `(vl-catch-all-error-stack OBJECT)` — a `clautolisp` extension — returns the captured stack of a caught error so AutoLISP code can inspect it programmatically. Both are described in the user manual §“`clautolisp` Extensions”.

11 File Compatibility

`clautolisp` honours the spec-derived AutoLISP path-resolution layer (an explicit current directory, support paths, trusted paths) rather than CL’s pathname merging. The behavioural details are exercised by the declarative file-compat scenarios under `clautolisp/autolisp-file-compat/scenarios/` and run by `make -C clautolisp run-file-compat-{sbcl,ccl}`. The scenarios include byte/text/newline roundtrips, encoding handling, mutation operations and printer/read-back loops, and form the canonical behavioural spec for the file-related builtins.

12 Building the Executable

From the `clautolisp/` subproject root:

```
make build-clautolisp-sbcl # produces tools/clautolisp/bin/clautolisp-sbcl
make build-clautolisp-ccl  # produces tools/clautolisp/bin/clautolisp-ccl
```

Both targets reuse the `read-autolisp` build harness. The SBCL build sets `:save-runtime-options t` so the saved core forwards every command-line argument to `clautolisp` rather than letting SBCL intercept `--help` or other runtime flags.

The build prerequisite chain (the source files that, if newer than the binary, trigger a rebuild) is captured by the `CLAUTOLISP_SOURCES` list in `autolisp-test/Makefile` when running the conformance suite.

13 User init files

On startup, `clautolisp` walks a fixed list of stems and loads every file that exists, in walking order. Earlier files are loaded before any user-supplied `-l` / `-x` / positional action, so the action queue runs against a fully-initialised evaluation context.

13.1 Stem list

#	Stem	Bare allowed?
1	<code>~/.autolisp{,rc}{,.lsp,.fas,.des,.vlx}</code>	yes
2	<code>~/.config/autolisp/init{,.lsp,.fas,.des,.vlx}</code>	no
3	<code>~/.clautolisp{,rc}{,.lsp,.fas,.des,.vlx}</code>	yes
4	<code>~/.config/clautolisp/init{,.lsp,.fas,.des,.vlx}</code>	no

Slots 1 and 2 are shared with the sibling `alfe` binary; a user who maintains a single `~/.autolisp` file sees it loaded by both. Slots 3 and 4 are specific to `clautolisp` and run **after** the shared pair — so program-specific definitions override the shared ones. Within each scope, the legacy HOME slot loads first and the XDG slot (`~/.config/<program>/init`) loads second, so a `~/.config/clautolisp/init.lsp` wins over both `~/.clautolisp` and the shared pair.

13.2 Per-stem extension preference

For each stem:

- The bare stem (without an extension) wins outright if it exists AND the slot allows it (slots 1 and 3). The XDG slots (2 and 4) ignore the bare stem.
- Otherwise the picker considers `<stem>.lsp` and the compiled variants `<stem>.{fas,des,vlx}`. A compiled variant wins only if it is strictly newer than `<stem>.lsp` (so editing the `.lsp` source automatically retires a stale compile). Among qualifying compiled variants, the newest wins.
- Falls through to `<stem>.lsp` if it exists and no compiled variant qualified.
- The stem is silently skipped if no file matches; missing init files are normal, not an error.

13.3 Gating

The lookup is gated by:

- `--no-init` or `-norc` on the command line — skip every stem.
- `$AUTOLISP_NO_INIT=1` (or `y` / `yes` / `true` / `on`, case-insensitive) — the shared kill-switch honoured by both `clautolisp` and `alfe`.
- `$CLAUTOLISP_NO_INIT=1` — disables init files for `clautolisp` only.

The CLI flag wins over the env vars; if either flag form is present the env-var rung is not consulted.

14 Versioning

The version string is stored at `clautolisp/tools/clautolisp/source/version.lisp` in `clautolisp.tools.clautolisp:*version*`. Format: `MAJOR.MINOR.DEVELOP`.

The `DEVELOP` counter is bumped on every change that touches `clautolisp` source code; the running binary's `--version` output is therefore a stable identifier of the build.

15 Conformance

`clautolisp` aims at the strict shared standard of the AutoLISP Spec by default (`--strict`), with vendor-specific behaviour selectable through `--autocad` and `--bricscad`. The conformance test corpus that exercises this is under `autolisp-test/` and is invoked via the `autolisp-test/clautolisp-driver` ASDF system or via the standalone-binary path (`make -C autolisp-test test-clautolisp-sbcl` and similar). Conformance reports for each binary are written under `autolisp-test/results/clautolisp/<version>/<host>/<timestamp>/`.

16 See Also

- `user-manual.org` — narrative end-user manual for the same binary, including REPL examples, banner format, and the `clautolisp` extensions documentation.
- `implementation-roadmap.org` — forward-looking architecture plan (Host Abstraction Layer, MockHost / GuiHost / LiveHost, per-phase decomposition).

- `development-plan.org` — high-level architectural rationale.
- `PLAN.md` — actionable task tracker for the implementation.
- `../../autolisp-spec/` — AutoLISP / Visual LISP language reference.
- `../../autolisp-test/` — conformance test suite that exercises `clautolisp-sbcl` and `clautolisp-ccl`.